

Counting GloBiMaps - A Probabilistic Data Structure for Handling Big Point Datasets

Moritz Laass

Technical University of Munich
TUM School of Engineering and Design
Munich, Germany
moritz.laass@tum.de

Martin Werner

Technical University of Munich
TUM School of Engineering and Design
Munich, Germany
martin.werner@tum.de

ABSTRACT

In this paper, we define a probabilistic data structure based on a layered construction built from Counting Bloom Filters and analyze the properties of this construction on real-world datasets. The key idea of the approach is to design a series of layers, each of which can accommodate a certain number of identical points. By taking a first layer to incorporate only small numbers, but for a comparably large number of different input values, the long tail of spatial data distributions can be well-served. Those slots of the first layer that are forced to overflow due to hash collisions or frequent items in the input distribution are transferred to a next layer of different characteristics. Multiple layers can be stacked and the spatial resolution can be modified to further increase the flexibility of the data structure. We show that basic Bloom filter operations such as the union and intersection can be lifted to this construction and show that the construction can be used to count data points in polygons or other complex geometries, without the need for unpacking. Case studies on map data from OpenStreetMap and social media data from Twitter illustrate the real-world behavior of this data structure.

PVLDB Reference Format:

Moritz Laass and Martin Werner. Counting GloBiMaps - A Probabilistic Data Structure for Handling Big Point Datasets. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/mlaass/counting-globimaps>.

1 INTRODUCTION

The management of large datasets has been a quite vibrant area of research in the last decades. Organizational schemes based on tree orderings of data spaces dominate the field (e.g., R-trees, ball trees) for their excellent performance and wide availability. However, it has also been realized that tree access methods have their limitations especially in parallel and distributed contexts. Locality sensitive hashing (LSH) and - in general - more random data distribution methods have taken the lead for data that is growing

beyond a practicable size for consistent query execution. In this context, Bloom filters play a vital role both in the coordination of nodes as well as in the context of log merge trees and key value stores. In these, Bloom filters are widely used to quickly assess if a certain block of data (e.g., a sorted string table on disk, a node in a distributed system, etc.) contains information about a certain key.

In this context, Bloom filters have been used literally as filter algorithms: they provide a compressed representation of what there might be in a data block and can reduce irrelevant I/O operations significantly with a small memory overhead. In this scenario, Bloom filters are optimized to be extremely small and fast: each object shall be modeled with only few bits and only very few hash function computations.

In previous work, we have investigated if and how Bloom filters can be used to directly model interesting datasets going for configurations with up to 32 GB of memory assigned for a single Bloom filter. We used Bloom filter for modeling huge, but sufficiently sparse geospatial raster datasets [24, 25] and trajectories of moving objects [23].

In this paper, we present a data structure Counting GloBiMap (CGM), which extends on classical GloBiMaps [24] by replacing Bloom filters with Counting Bloom filters of varying bit depth. Due to the power law structure of typical datasets, the discretization inherent to GloBiMaps leads to few objects with extremely high observation frequency in input data (e.g., locations with a high number of messages) and the majority of locations with moderate to small numbers of data items associated.

In order to deal with this data distribution, we first choose a small number of bits for the counting bloom filter, such that many of the involved bits will overflow during data intake. In this context, the data structure uses a multilayer approach. The data structure is configurable to hold as many layers as necessary, whereas each layer basically represents a counting Bloom filter with a fixed size and bit-depth and the insertion of a new element will increment the first non-maximal layer of the construction. That is, in the beginning, all insertions increment locations in the first filter. As soon as a few slots of the first layer reach their maximal value of an all-one bit string, the insertion is forwarded to the second layer.

This setup allows for a pyramid like construction of the multiple layers, with the lower layers being of bigger sized filters with lower bit-depth per slot, while the higher layers are comprised of smaller sized filters with higher bit-depth. While there have been approaches in Bloom Filters, that take the distribution of the to be contained data into account [10] [6]. The way our data structure operates has not been proposed so far. Additionally, the simplicity of

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

our approach also holds promise to enable a straight forward implementation even in reconfigurable devices and hardware [8, 12, 21]. In addition, we expect predictable performance, as bit-depths that are native to the hardware environment and fixed size filters will in most cases lead to better performance and memory layouts than some of the approaches in the literature, that rely on dynamic resizing of filters and bin sizes [11].

Another interesting fact about the data structure is the possibility of performing certain operations using the compressed representation without the need to unpack the data into a more classical representation, such as a bitmap or (sparse) matrix. We show that this property might be useful to approximate some types of spatial queries, for example counting points in a polygon. While the approximation of a query in such a probabilistic setting is interesting in itself, we envision additional help for query planning as these data structures give access to frequency information and expected candidate set or result set sizes as well. This could either be used to stop queries that contain a too large result set, or to decide about sub query priorities and distribution strategies, which has been successfully proposed using simple Bloom filters [14, 16, 19].

The remainder of the paper is structured as follows: in Section 2 we shortly review previous work related to our research. In Section 3, a thorough overview of the data structure and its properties is given. In Section 4, a set of operations on Counting GloBiMaps is introduced, which can be used to model certain types of queries. In Section 6, an experimental evaluation on real world spatial datasets is presented, documenting the behaviour and performance of our data structure. Section 7 concludes the paper.

2 RELATED WORK

In this section, we discuss Bloom filters which have been invented to represent extremely sparse subsets of strings, such as English words with irregular spelling and have since been used in various applications. Further, we examine Counting Bloom filters which are an extension to Bloom filters allowing for a scalar representation of entries instead of a binary one. Despite the fact that Bloom filters and Counting Bloom filters have been frequently used in big data systems, it has not yet been applied to represent sparse frequency rasters. However standard Bloom filters have been used to represent small collections of locations in web browsers [22] as well as trajectories [23] and geospatial bitmaps in the previous work on globimaps [24]

2.1 Bloom Filters

The Bloom filter data structure was introduced by Bloom in 1970. It is a probabilistic data structure capable of encoding sets in main memory [4]. It gives the option of deciding how much memory to utilize to model the set, trading memory consumption and compute effort with error probability. However, the most essential feature of Bloom filters is that they only have one type of errors: false positives. While the data structure representing a set will never forget an element that has been added to the set, it may report elements that have never been added to the set as belonging to the set. But this error can be kept arbitrary small by investing more computation and memory. The Bloom filter’s basic structure is as follows:

One starts by defining F as an m -bit array. We then use k pairwise independent hash functions h_i that map all of the set’s potential elements to a non-negative integer value less than m . An all-zero Bloom filter array F represents the empty set. The Bloom filter now has two operations: Insert and Test, which expose its capabilities. The function Insert alters the Bloom filter to express the fact that an element has been introduced, given an element e . Given the limitation of false positive findings, the function Test reports whether the data structure suggests that the element has been entered before. The value of the specified k hash functions applied to the provided element e is calculated first in both processes. The Insert function sets all of the slots designated by the hashing results to one, but the Test function indicates that the element is not in the set if one of these outcomes refers to a zero slot. The Insert operation sets the same bits to one that the Test function searches for a zero, therefore there can’t be any false negatives in this data structure. When all of the slots indexed by hash functions are filled with ones, on the other hand, we can assume that the element has been entered, even though the values could have originated from a combination of adding distinct elements due to probable hash collisions. Note that the Insert operation will always compute all hash values and access all indexed locations while the Test operation is capable of early stopping on the first zero encountered.

It’s fascinating to see how errors in this data structure can be modeled. A Bloom filter with n elements can be viewed as the consequence of setting an essentially random bit in the underlying array to one for kn times, with the hash functions transforming the elements into uniformly distributed random integers. Finding a false positive is now as likely as finding k ones in random filter slots, which is as likely as finding a one k times in a random slot. In fact, the following formula connects the parameters m , k , and the number of inserted elements n to a fraction of zeros measure (foz).

$$\text{foz} = \left(1 - \frac{1}{m}\right)^{kn} \approx e^{-\frac{kn}{m}} \quad (1)$$

This fraction of zeros is a suitable proxy for the likelihood of discovering a zero in a random location assuming the hash functions are sufficiently random and independent (thus also assuming sufficiently large filters). As a result, the filter’s false positive probability can be estimated as follows: If k random accesses to the bit field discover a one, a false positive occurs.

$$p = (1 - \text{foz})^k \approx \left(1 - e^{-\frac{kn}{m}}\right)^k \quad (2)$$

We may search for a minimum of k by differentiating this equation and locating the roots because the right hand side is differentiable. This leads to the conclusion, that

$$k_* = \frac{m}{n} \ln 2 \quad (3)$$

is the (anticipated) false positive rate’s global minimum. That is, we can determine an optimal value k_* given a memory budget of m bits and a number of components n to be fed into the filter. However, because k is required to be an integer number of hash functions, we will need to round k_* to the nearest integer. Furthermore, m is often constrained to memory pages or even powers of two to allow for faster modulo computation and the value of n is rarely known beforehand [5].

2.2 Counting Bloom Filters

Many papers on BF have been published, with compressed BFs [17], distance-sensitive BFs [13] dynamic BFs [11], being some of the most important innovations in the area of BFs.

As previously noted, BFs are strictly binary in the sense that they can only tell if an item is in the set or not, but not how many items of the same value have been added, nor do they allow for the removal of items from a set in a sensible way. Towards this end, Counting Bloom Filters (CBFs) were introduced by Fan in 2000 [9] which use small unsigned integer bins rather than single bits of presence. The appropriate bin counters are incremented when an item is inserted or decremented when removed. While CBFs have many desirable properties, they introduce the issue of counter overflow, which must be considered when designing the data structure. Many enhancements to CBFs have been made in attempt to overcome these constraints and to gain improved performance. And the amount of main memory is now significantly higher, especially when common CPUs with a smallest addressable unit of 8 bits are used. It is very tricky to efficiently implement 3-bit counters on 64 bit CPUs.

DCF (Dynamic Count Filters) [2] are basic data structures designed for speed of retrieval and adaptability. They do not require the usage of indexes like other schemes, thus resulting in fast access times and the ability to store arbitrary amounts of data.

DCFs are made up of two vectors: one is a standard CBF with fixed-size counters, and the other is an Overflow Counter Vector with a counter for each element of the first vector that keeps track of the number of overflow occurrences. To avoid saturation, the size of counters in overflow counter vector is changed dynamically. This of course means that each update requires rebuilding of the data structure, which comes with a significant performance overhead for insertions, which is different from our approach. Other schemes such as Multilayer Compressed Counting bloom filters are utilising Huffman codes to optimize bin size and have introduced the idea of a hierarchical multilayer structure [10].

2.2.1 Binary GloBiMaps. In a previous paper Global Binary Maps (GloBiMaps) were introduced using binary Bloom filters to represent global raster data sets in the spatial domain. It was shown that many desirable properties of 2D raster datasets still hold true in this representation such as low resolution incremental rendering, the ability to form quadrees using alternating coding, low memory overhead error correction and fast compression using the mod-2 strategy [24].

For GloBiMaps, a global sparse raster (e.g., all human built-up area in the world) is used and the integer raster coordinates are hashed in order to write into a Bloom filter. An additional error correction table makes sure that false positives of the structure are corrected. Experiments show that extreme compression ratios are possible and common for certain sufficiently sparse datasets. Furthermore, the false positives have been interpreted as the addition of a uniform random noise to the underlying truth and we were able to remove a huge part of this noise by training a convolutional neural denoiser in residual representation [25]. In this way, we were able to accommodate a higher amount compression ratios efficiently, but at the cost of allowing false negatives introduced by the denoising scheme.

3 COUNTING GLOBIMAPS

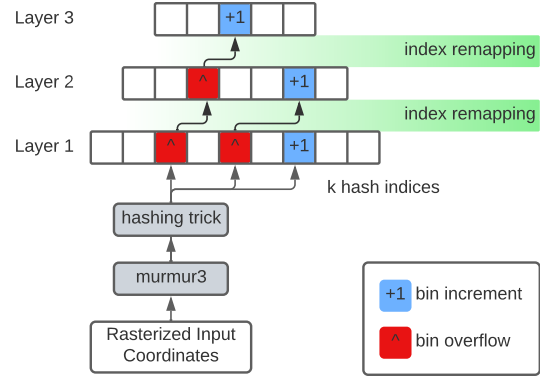


Figure 1: CGM MultiLayer Structure

Figure 1 depicts the construction of Counting GloBiMaps in the context of managing spatial point cloud datasets or frequency rasters. Similar to GloBiMaps, we start with spatial input data points such as messages or events. In a first step, we discretize the coordinates of the spatial objects (e.g., we rasterize them) and apply a combination of a good and fast hash function (Murmur3 in our implementations) and the hashing trick to generate values of k sufficiently independent uniform hash values. These hash values are now used to address an array of small counters as in the Counting Bloom filter construction descending into higher layers in case of overflow.

More precisely, a Counting GloBiMaps consists of a set of k hash function and a set of Λ layers L_x each consisting of a filter of size m_x with bins of bit depth b_x .

$$CGM = \{k, L_x = \{m_x, b_x\}\} x = 1.. \Lambda$$

For a bit-depth b_x , the threshold value T_x is computed as

$$T_x = 2^{b_x} - 1$$

The bins of the first layer are incremented until they reach the threshold value T_1 . Then, the corresponding bin of the second layer is incremented. This scheme is then repeated for subsequent layers.

To access the value of a bin on a specific layer of a CGM A we introduce the function $\text{val}(i, L_{A,x})$ which retrieves the value of the i -th bin of layer L_x of CGM A .

To generate k hash functions, the implementation of CGM uses the hash function **murmur3** and the well known hashing trick, by which a single call to the the function can be used to generate the series of k hash functions [25]. The implementation of **murmur3** takes a 128 bit field as input and returns a 128 bit field as output. In our implementation the input is generated by concatenating the 2 components of a 64 bit unsigned integer coordinate c . And the output field is split into two 64 bit unsigned integers h_1 and h_2 .

To complete the description of the data structure, we introduce the two operations PUT and GET which insert a point into the data structure or retrieve the count of elements from the data structure. Note that the count can be higher due to false positives. We omit

the description of subtraction, because it has little practical value and does not need an additional idea. Just decrement the highest available slot.

- (1) *PUT Operation* The PUT operation adds a discrete point to the CGM by incrementing the associated bins values
- (2) *GET Operation* The GET operation retrieves a discrete point's value from the CGM by finding the smallest value in the associated bins

3.1 The PUT Operation

The PUT operation is adding a discrete point to the CGM by incrementing the filter bins corresponding to the point starting in the first layer. For every bin, the overflow is handled by incrementing the corresponding bin in the next layer until there is no more overflow or the last layer is reached which means there are no more layers to overflow too.

In Listing 1 the algorithm for the PUT operation is described, notable here is the hash function **murmur3**, which takes as input a coordinate c consisting of 2 64 bit unsigned integer values and returns as output h_1 and h_2 which are 64 bit unsigned integers. The bin index j is created using the hashing trick and the modulo operation mod with the layer size m_x as operand. In practice another trick was used here, with layer sizes restricted to powers of two the modulo operation can be implemented with a simple bitwise AND operation on a mask consisting of $\log_2(m_x)$ ones.

Algorithm 1: PUT Operation

```

Input: CGM  $A = \{k, b_x, m_x\} x \in 1 \dots \Lambda$ , point  $p$ 
 $c \leftarrow \text{RasterProjection}(p)$ 
 $h_1, h_2 \leftarrow \text{murmur3}(c)$ 
foreach  $i \in 0 \dots k$  do
    foreach integer  $x \in 1 \dots \Lambda$  do
         $j \leftarrow \text{mod}(h_1 + (i + 1) * h_2, m_x)$ 
         $v \leftarrow \text{val}(j, L_{A,x})$ 
        if  $v < 2^{b_x} - 1$  then
             $\text{val}(j, L_{A,x}) \leftarrow v + 1$ 
            break
        end
    end
end

```

3.2 The GET Operation

The GET Operation is retrieving the minimal value found for a discrete raster coordinate c . As described previously the hash function **murmur3** is used in conjunction with the the hashing trick and the modulo operation to generate bin indices.

In this algorithm, to retrieve the minimal bin value v_{min} , it is initialized as positive infinity in practice this is the largest possible integer size for the given return type. The algorithm iterates over all bins either stopping early if it retrieves a zero, iterating over all layers that match the threshold value or using the break instructions to avoid iterating over empty layer bins. In case of overflows the bin values of the corresponding layers are summed up in the variable s . For each k v_{min} is derived by applying the min operation comparing

v_{min} with the acquired sum s . Finally the smallest bin value v_{min} is returned as result.

Algorithm 2: GET Operation

```

Input: CGM  $A = \{k, b_x, m_x\} x \in 1 \dots \Lambda$ , point  $p$ 
Result:  $v_{min}$ 
 $c \leftarrow \text{RasterProjection}(p)$ 
 $h_1, h_2 \leftarrow \text{murmur3}(c)$ 
 $v_{min} \leftarrow \infty$ 
foreach  $i \in 0 \dots k$  do
     $s \leftarrow 0$ 
    foreach  $x \in 1 \dots \Lambda$  do
         $j \leftarrow \text{mod}(h_1 + (i + 1) * h_2, m_x)$ 
         $v \leftarrow \text{val}(j, L_{A,x})$ 
         $s \leftarrow s + v$ 
        if  $v = 0$  then
            Result: 0
        end
        if  $v < 2^{b_x} - 1$  then
            break
        end
    end
     $v_{min} \leftarrow \min(v_{min}, s)$ 
end
Result:  $v_{min}$ 

```

Notable in this implementation is the early stopping both for encountered 0 value bins and for bins that are below threshold value. In practice this means, that querying CGMs that are relatively more "empty" should be faster, meaning that we should see slower query performance when foz decreases. Another, less pronounced slowdown should be observable for CGMs and coordinates with lots of overflow.

3.3 Probability of False Positives and Error Sizes

As previously introduced the probability of false positives for a standard BF is

$$p = \left(1 - \left[1 - \frac{1}{m}\right]^{kn}\right)^k \approx \left(1 - e^{-\frac{kn}{m}}\right) \quad (4)$$

where n is the number of elements added to the BF and m the size of the BF. In the case of CBF and therefore CGM, n is to be understood as the number of unique positions and we introduce t as the total number of elements. With this distinction we can make an assumption about the total size of the error, given by the number of false positives multiplied by the average increase in value given by the ratio $\frac{t}{n}$ times the number of hash functions.

$$\text{err}_{total} = k \frac{t}{n} \left(1 - e^{-\frac{kn}{m}}\right) \quad (5)$$

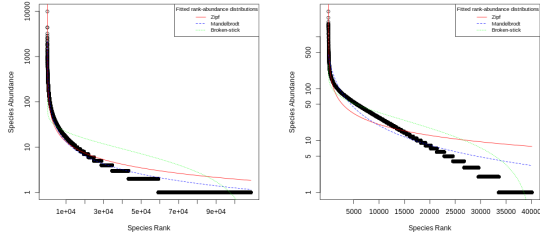
With the total error given, we can derive the mean error simply with a simple division:

$$\text{err}_{mean} = \frac{\text{err}_{total}}{n} = k \frac{t}{n^2} \left(1 - e^{-\frac{kn}{m}}\right) \quad (6)$$

While such a construction might not be useful for many use cases dealing with arbitrary data distributions, the spatial case is

often following a power distribution such that we can have some assumptions about $\frac{t}{n}$. At this point, the intuition can be gained, that due to large frequencies t falling into a small frequencies of n the case emerges where the error sizes significantly increase for only small portions of the data, specifically those bins with large values. This is one of the motivations for a modification of the CGM construct given in Section 5 in which a re-randomization between all layers reduces the impact of degenerating uniform randomness with increasing layer index.

3.4 Optimal Configuration Procedure



(a) Distribution of the Twitter dataset (b) Distribution of the OSM dataset

Figure 2: Distributions of different datasets with fitted distribution curves

To have a discussion of optimal configurations for CBF, obvious topics are the optimum number of hash functions k , the optimum size of the filter m and the bit depth b for each layer. For CGMs the complexity is multiplied by the number of Layers L which need to be configured with a size m_x and bit depth b_x .

To make an informed decision about these parameters one aspect that is important is the (assumed) distribution of the data to be encoded. One way this can be approached is by modeling the distribution, using a well-known distribution such as the Zipf distribution [26], the Mandelbrot distribution [15], or the Broken-stick distribution [3]. Such distributions can be (partially) fit to a subset of the data, for example with machine learning techniques [20].

As depicted in Figure 2, the aforementioned models definitely don't fit our discrete and finite data, but their behavior seem to be a good model of the real-world data.

Starting from an assumption of how many data items are going to be modeled, one can now start to search the configuration space as follows: one needs to introduce a first layer which will receive all of the data. That is, one can sample from the given distribution and perform or simulate the insertion. It is important to now extract the overflow data items which would go to the second layer.

If one chooses a small bit depth and a moderate filter size, most of the slots will overflow and it is clear that the data distribution of overflowed data will be very similar to the previous layer. Only a few very rare instances from the long tail might be absorbed. If, however, one chooses a comparably large filter size or larger bit depths or a combination of both, only the very frequent data items will overflow and, hence, the higher layers will see discrete distributions.

The real behavior can be anything between. Still, the take away is (1) that an optimal configuration is not possible as one cannot derive the statistical properties of the output distributions of the layers in general, and (2) that the number of different data items in higher layers decreases such that, as a consequence, the uniform distribution assumption of the Bloom filter theory gets broken.

4 OPERATIONS ON COUNTING GLOBIMAPS

Counting GloBiMaps can be interpreted as representation of point-clouds on grids, which themselves can be representations of various spatial structures. This section describes some meaningful operations for spatial databases and gives hints on their precision. The compressed representation combined with the expressiveness of some of these operations make it feasible to perform ad-hoc approximations of certain spatial queries and to perform calculations on clients and edge devices on the fly without ever needing to query for the uncompressed input data.

- (1) **UNION Operation** creates a union CGM by adding two CGMs
- (2) **INTERSECTION Operation with MERGE function** creates an intersection CGM by merging the overlapping points of two CGMs using a merge function
- (3) **SUM Operation** calculates a sum of an area given by a list of hash values

4.1 Union and Intersection

The Counting GloBiMaps structure as an extension of GloBiMaps and therefore of BFs has several interesting properties, which shall be discussed here. BFs as representations of sets can perform operations that are familiar from set theory such as intersections and unions. In a binary bloom filter these can be expressed using a simple binary operation such as bitwise OR and bitwise AND directly on the bloom filter data.

When using Counting Bloom Filters this is a bit less straight forward to implement and for Counting GloBiMaps the layered structure needs to be taken into account. Nonetheless, these operations can be quite useful for various applications therefore algorithms for implementing these operations will be presented along with an analysis of the algorithms complexity.

We shall also note the special case of a Counting GloBiMap that has a binary filter as its first layer (equivalent to a bit size of 1) which can exploit the simplicity of the binary operations for the first layer as known from BFs and GloBiMaps [24].

To understand the following algorithms, the insert and get operations are defined first to aid notation.

- **get**(A, i) takes as input a CGM A and a bin-index i , it retrieves the value from the bin indexed by i , escalating and summing up the values of higher layers when it encounters threshold values concordant to the algorithm of the GET operation in Listing 2
- **insert**(A, i, v) takes as input a CGM A and a bin-index i and a value v , it inserts the value into the CGM similarly to the PUT operation described in Listing 1, in this case avoiding the need to call the hash function as the bin index is given.

4.2 Union Operation

One way to interpret Counting GloBiMaps is as a Vector of n dimensions, with n being size of the largest layer. With this definition the Union of two CGMs is a simple vector addition with the simplest implementation for a single layer 1 bit CGM being a bitwise OR operation. The Union allows for merging two disparate datasets into one CGM.

Algorithm 3: Union Operation

Input: CGM $A = \{k, b_{A,x}, m_{A,x}\}$
 $B = \{k, b_{B,x}, m_{B,x}\} x \in 1 \dots \Lambda m_{A,1} \geq m_{B,1}$
Result: CGM I
 initialize I with 0
foreach $i \in 0 \dots m_{A,1}$ **do**
 if $\text{val}(\text{mod}(i, m_B), L_{B,1}) \neq 0$ **OR** $\text{val}(i, L_{B,1}) \neq 0$
 then
 $s \leftarrow \text{get}(B, i) + \text{get}(A, i)$
 insert(I, i, s)
 end
end
Result: I

The worst case complexity of the Union operation is:

$$O(\sum_{l=1..L} m_l) = O(L\overline{m_l}) \approx O(Lm_1)$$

4.3 Intersection Operation Family

When lifting the Intersection Operation to CGMs we have the opportunity to introduce a family of operations, derived from the classical intersection, which for single bit BFs would be the bitwise AND. For CBFs it would be an elementwise minimum operation. In our construct we extend the notion of the elementwise operation by introducing a merge function $\text{merge}(v_a, v_b)$ taking the the value of th the Operands elements as input thereby generalizing the idea, that is implicit in the intersection operation in order to arrive at a set of operations known from computer graphics.

Algorithm 4: Intersection Operation Family with textbfmerge function

Input: CGM $A = \{k, b_{A,x}, m_{A,x}\}$
 $B = \{k, b_{B,x}, m_{B,x}\} x \in 1 \dots \Lambda m_{A,1} \geq m_{B,1}$
 $\text{merge}(v_a, v_b)$
Result: CGM I
 initialize I with 0
foreach $i \in 0 \dots m_{A,1}$ **do**
 if $\text{val}(\text{mod}(i, m_B), L_{B,1}) \neq 0$ **AND** $\text{val}(i, L_{B,1}) \neq 0$
 then
 $v \leftarrow \text{merge}(\text{get}(B, i), \text{get}(A, i))$
 insert(I, i, v)
 end
end
Result: I

The worst case complexity of the Intersection operation is:

$$O(\sum_{l=1..L} m_l) = O(L\overline{m_l}) \approx O(Lm_1)$$

In the context of the union information, different slot values need to be aggregated taking into account the current values in the same slots of two different CGMs. The usual approach would be to take the sum of the values of the counters. This mimics the behavior of a filter in which all elements from both CGMs have been inserted. As mentioned before it is also possible to define other merge functions, for example minimum, subtract, difference, and mask. Especially the non-symmetric mask operation is interesting as it models the subset of A that overlaps with B . As B can be a single layer, single-bit GloBiMap, we can use this operation to perform basic spatial statistics on the compressed data representations in a CGM with respect to a GloBiMap as described in Section 4.4. In the following some possible merge functions are reviewed.

- **minumim**(v_a, v_b) = $\min(v_a, v_b)$ Minimum implements a classical intersection 4.4.
- **mask**(v_a, v_b) = v_a subset of A that overlaps with B useful for spatial statistics as described in Section
- **add**(v_a, v_b) = $v_a + v_b$ Adds the values where a spatial intersection occurs. This is is useful for aggregating results of different mask operations
- **subtract**(v_a, v_b) = $\max(v_a - v_b, 0)$ Subtracts the values where a spatial intersection occurs. This is is also useful for aggregating results of different mask operations
- **difference**(v_a, v_b) = $|v_a - v_b|$ takes the absolute difference between values. This is is also useful for aggregating results of different mask operations, when subtract function would lead to zero result values

4.4 Spatial Statistics

In many tasks on geometric data, one is interested in spatial summaries such as how many objects fall into a given polygon or what is the average temperature measured in a city. To query a CGM for the sum of points in a given area, one can use the GET Operation over a suitably rasterized geometric region. A sensible optimization of such an operation, especially if an area will be reused over time, is to cache the hash function values h_1 and h_2 for each point of the rasterized geometry. This leads us to the SUM Operation described in the Listing 5

Depending on the constraints of the hardware and the number of polygons to cache this may or may not be a feasible strategy.

Another possible optimization especially useful if there are many and overlapping polygons is to encode the polygons in single layer Binary GloBiMaps one for each polygon along with a bounding box covering the polygon bounds and caching the hash functions for the whole possible 2D space. Then the sum can be computed by first applying the Intersection Operation with mask merge function and in a second step performing the SUM operation, using the cached hash functions covering the bounding box. Such complex combinations of stages that share similarities with sprite graphics can be surprisingly powerful, but are beyond the scope of this paper.

5 MULTIREOLUTION COUNTING GLOBIMAPS

Quite a few times, we have already remarked that something bad is happening to the distribution properties from layer to layer. By truncating away the less frequent data instances layer by layer, the

Algorithm 5: SUM Operation

Input: CGM $A = \{k, b_{A,x}, m_{A,x}\} x \in 1 \dots \Lambda$ list of hash values $h_{j,y}, y \in \{1, 2\}, j \in 0 \dots N$

Result: z

```

 $z \leftarrow 0$ 
foreach  $j \in 0 \dots N$  do
   $v_{min} \leftarrow \infty$ 
  foreach  $i \in 0 \dots k$  do
     $s \leftarrow 0$ 
    foreach  $x \in 1 \dots \Lambda$  do
       $j \leftarrow \text{mod}(h_{j,1} + (i+1) * h_{j,2}, m_x)$ 
       $v \leftarrow \text{val}(j, L_{A,x})$ 
       $s \leftarrow s + v$ 
      if  $v < 2^{b_x} - 1$  OR  $v = 0$  then
        break
      end
    end
     $v_{min} \leftarrow \min(v_{min}, s)$  if  $v_{min} = 0$  then
      break
    end
  end
   $z \leftarrow z + v_{min}$ 
end
Result:  $z$ 

```

variability of the data in higher layers is decreasing and this can lead to degeneracies, for example, when the hash functions are not getting sufficiently diverse input to surjectively cover and use the allocated slots. We therefore suggest a new strategy of remapping indices between layers increasing the randomness from layer to layer to counter the inherent decrease of variability. Keeping the CGM layer structure, instead of remapping the indices that get propagated with a simple modulo operation a new remapping technique involving discretization of the coordinates at a higher resolution is introduced.

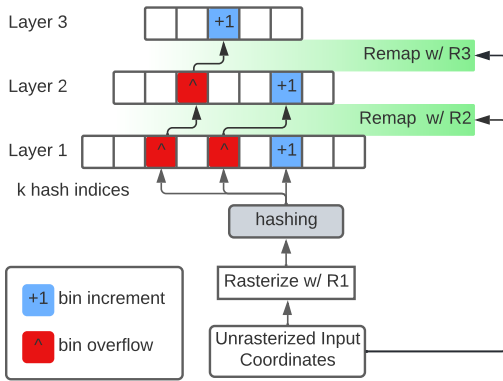


Figure 3: CGM Multi Resolution Setup

Therefore the CGM definition is extended with a resolution for each layer. To simplify the discussion, let us assume (without loss of generality), that all input coordinates are scaled into the range

$[0, 1]$. Then, the resolution can be given by an integer number r_x defining the cell division index to be the the coordinate multiplied by this resolution coefficient rounded down to the nearest integer.

$$CGM = \{k, L_x = \{m_x, b_x, r_x\}\} x = 1 \dots \Lambda$$

What happens in this setting is that coordinates that are mapped to the same cell in low resolution will be re-mapped at higher resolution hoping that they might lead to different high-resolution coordinates and therefore, increased variability in the subsequent layer. Depending on how the resolutions are used, one can generate a quadtree like refinement structure such that the first layers of such Counting GloBiMaps can be of rather low resolution expecting that they will be subdivided as soon as they get visited often enough thereby becoming more interesting. However, such an approach differs significantly from common raster data management as the subdivision occurs locally only.

In the following we will describe the modified PUT and GET operations.

Algorithm 6: Multiresolution PUT Operation

Input: CGM $A = \{k, \{b_x, m_x, r_x\}\} x \in 1 \dots \Lambda$

Coordinate c

$c_1 \leftarrow c \cdot r_1$

$h_1, h_2 \leftarrow \text{murmur3}(c_{r_1})$

foreach $i \in 1 \dots k$ **do**

foreach $x \in 1 \dots \Lambda$ **do**

if $x > 1$ **then**

$c_x \leftarrow c \cdot r_x$

end

$j \leftarrow \text{mod}(h_1 c_{x,1} + i h_2 c_{x,2}, m_x)$

$v \leftarrow \text{val}(j, L_{A,x})$

if $v < 2^{b_x} - 1$ **then**

$\text{val}(j, L_{A,x}) \leftarrow v + 1$

break

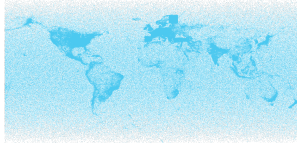
end

end

end

The multiresolution PUT operation described in Listing 6 is similar in structure to the PUT Operation described in Listing 1, but with the addition of calculating the resolution for each layer that is visited. Note that we can use the hashing trick and add the rasterized coordinate values as factors for h_1 and h_2 . Alternatively, one can calculate new hash values for each layer.

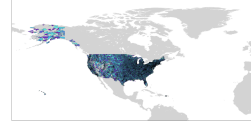
The multiresolution GET operation comes in a few different flavors which are necessary due to the fact, that the number of layers affected by a given coordinate is not predictable and therefore the resolution at which the data is held in practice is not known beforehand. Therefore we suggest to either query for a specific resolution layer in which case there are two possible outcomes: either the coordinate exists in the given resolution, which means it can be returned or it does not in which case the GET operation can either return a special state, such as -1 or it can return the the highest resolution value below the queried resolution. The other option is to query without asking for a specific resolution in which case there are two possible result strategies: either the



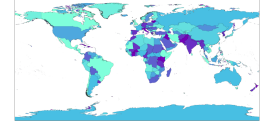
(a) Scatterplot for the first 100 mio points of the Twitter dataset



(b) Scatterplot for the first 100 mio points of the OSM dataset



(c) US Census Zip Codes



(d) Global Administrative Boundaries

Figure 4: Pointcloud and vector datasets

operation returns the highest resolution it can find and returns it along with the resolution value. The other option is to return all found resolutions and return them along with the values in a container such as a list, array or hashmap.

Algorithm 7: Multiresolution GET Operation returning highest resolution

Input: CGM $A = \{k, b_x, m_x, r_x\} x \in 1 \dots \Lambda$
Coordinate c
Result: $v_{min} r_v$

$c_1 \leftarrow c * r_1$
 $h_1, h_2 \leftarrow \text{murmur3}(c_1)$
 $v_{min} \leftarrow \infty$

foreach $i \in 0 \dots k$ **do**
 $s \leftarrow 0$ $r_t \leftarrow 0$
foreach $x \in 1 \dots \Lambda$ **do**
 if $x > 1$ **then**
 $c_x \leftarrow c * r_x$
 end
 $j \leftarrow \text{mod}(h_1 c_{x,1} + i h_2 c_{x,2}, m_x)$
 $s \leftarrow s + \text{val}(j, L_{A,x})$
 $r_t \leftarrow r_x$
 if $\text{val}(j, L_{A,x}) = 0$ **then**
 Result: $0, r_1$
 end
 if $\text{val}(j, L_{A,x}) < 2^{b_x} - 1$ **then**
 break
 end
end
 $v_{min} \leftarrow \min(v_{min}, s)$ **if** $s = v_{min}$ **then**
 $r_v \leftarrow r_t$
end
end
Result: v_{min}, r_v

6 EVALUATION

This section shall describe the experiments that were performed in order to show the properties of CGM on datasets related to human activity on Earth. Because of the big amount of configurable parameters, e.g. amounts, sizes and bit-depth of layers, number of hash functions and different resolutions of the dataset discretizations, the experiments are structured in such a way as to attempt to isolate the behavior of each parameter.

Algorithm 8: Multiresolution GET Operation with resolution query

Input: CGM $A = \{k, b_x, m_x, r_x\} x \in 1 \dots \Lambda$
Coordinate c
layer x **Result:** v_{min}

$c_1 \leftarrow c * r_1$
 $h_1, h_2 \leftarrow \text{murmur3}(c_1)$
 $v_{min} \leftarrow \infty$

foreach $i \in 0 \dots k$ **do**
 $s \leftarrow 0$
 if $x > 1$ **then**
 $c_x \leftarrow c * r_x$
 end
 $j \leftarrow \text{mod}(h_1 c_{x,1} + i h_2 c_{x,2}, m_x)$
 $s \leftarrow s + \text{val}(j, L_{A,x})$
 $r_t \leftarrow r_x$
 if $\text{val}(j, L_{A,x}) = 0$ **then**
 Result: $0, r_1$
 end
 $v_{min} \leftarrow \min(v_{min}, s)$
end
 $v_{min} \leftarrow v_{min} + \sum_{i=1}^x t_i$
Result: v_{min}

6.1 Datasets

Two pointcloud datasets were used in this experiment; the first is a collection of twitter locations and the second is a collection of Open Street Maps(OSM) point objects. With respect to the spatial statistics queries, polygon datasets of US zip codes as well as bounding polygons of all countries in the world have been used. Table 1 gives some characteristics of these datasets.

6.1.1 Global Twitter Dataset. The Twitter dataset consist of more than 270 million individual locations mined from Twitter. This dataset shows usage patterns with highest densities in North america and Europe [24]. It also features a certain percentage of random locations which are distributed across the globe, due to bot activity. This dataset has been subdivided by taking the first N points for values of 1 million, 10 million, 50 million, 100 million, and 200 million. Except for scaling and discretization which was applied in the following experiments in order to encode the dataset into a CGM, no further processing or data cleaning was applied.

6.1.2 OpenStreetMap (OSM) Dataset. OpenStreetMap is an editable map database built and maintained by volunteers [18]. In 4

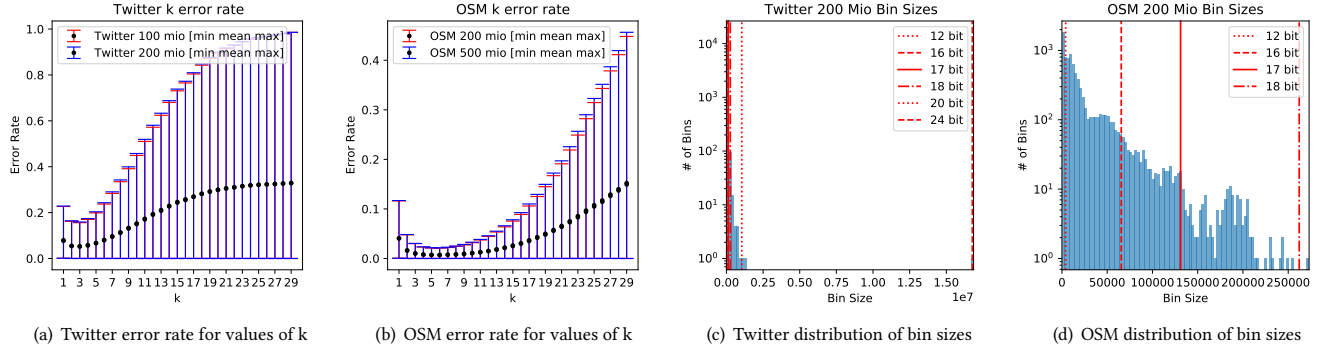


Figure 5: Pointcloud Dataset Properties

the OSM dataset is depicted, which is a collection of points derived from OSM data. It consists of more than 1.5 billion points that are located on the Asian continent. This dataset has been subdivided by taking the first N points for values of 200 million, 500 million and 1 billion. Except for scaling and discretization which was applied in the following experiments no further processing or cleaning was applied.

6.1.3 Polygon Dataset US Census Zip Codes. The US census dataset consists of 2017 Census Zip Codes Polygons, which are all in all more than 44 thousand individual polygons of various sizes. They cover the area of the United States, with higher density of polygons around more densely populated areas, which correlates with areas of higher density in the Twitter dataset. The dataset was downloaded from the United States Census Bureau website in the form of shapefiles. Apart from scaling and rasterization in various resolutions, no further processing was applied. [7]

6.1.4 Global LSIB Polygons Detailed. The LSIB Polygons Dataset has been downloaded in shapefile format from the Humanitarian Data Exchange. it consists of more than 4000 highly detailed polygons and is described as follows:

“The Office of the Geographer’s Global Large Scale International Boundary Detailed Polygons file combines two datasets, the Office of the Geographer’s Large Scale International Boundary Lines and NGA shoreline data. The LSIB is believed to be the most accurate worldwide (non- W. Europe) international boundary vector line file available.” [1] Other than scaling and rasterization in various resolutions, no further processing was applied.

6.2 Analyzing the Importance of Resolutions

When deciding on a resolution in which to rasterize points for the inclusion in Counting GloBiMaps, several aspects can be of importance. First the resolution will decide the actual spatial accuracy of points added to the data structure, which is highly application dependent and will not be discussed here. Beyond that, various properties that change given a different resolution, such as unique number of points and bin size distributions, that have an impact on fraction of zeros f_0 and expected error size as shown earlier. Table 2 presents an overview.

Looking at the Table 2, we can clearly see that the two datasets with same number of points have wildly different properties, whereas the OSM dataset at a resolution of 16, 384 already has a maximum bin size smaller than 2^{15} and can therefore be fully represented with bit-depths of 15 bits and smaller. The Twitter dataset at the same resolution has a minimum bit-depth of 24 bits but the number of unique points is not even half the amount in comparison. Interestingly the the median and 75th percentile for the Twitter dataset are much lower, which leads to the conclusion that the Twitter dataset has a few highly concentrated hotspot, whereas the OSM dataset has a more even distribution, which is confirmed by Figures 5. In such a case the multilayer architecture of CGMS can be exploited by adding a layer with a high bit-depth b_x relatively small filter size m_x at the top of the layer stack.

Dataset	Size	Optimal k	Encoding time		
			min	max	mean
Twitter	213,869,519 points	3	10.0824	16.0112	13.87
OSM	1,563,146,114 points	6	73.1999	98.5461	88.07
# of pixels @ 16384x16384					
Global Administrative Boundaries	4456 polygons	-	0	4.95708e+07	39849
Us Census Zip Codes	43,431 polygons	-	0	61710	161.876

Table 1: Properties of datasets

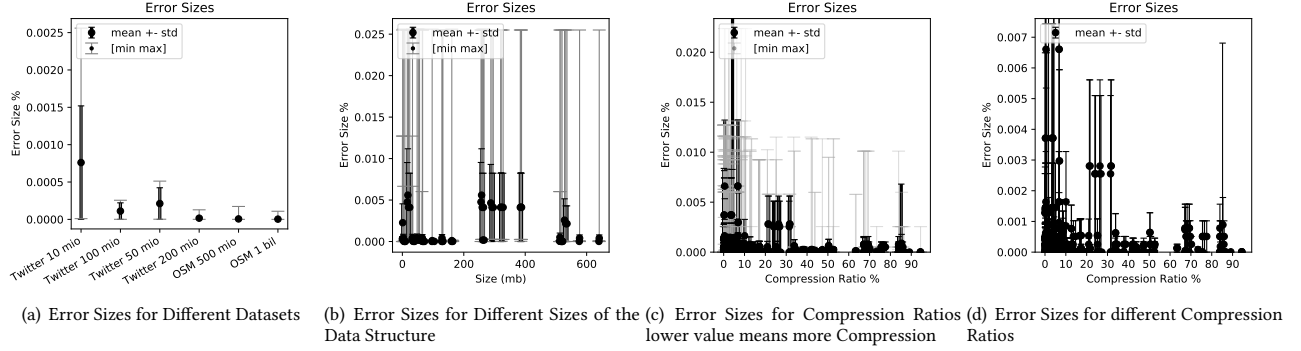


Figure 6: Error Sizes are in Relation to the Size of the dataset

6.3 Error Rate Distribution for Different Values of k

In order to analyze the optimum number of hash functions k for a given dataset, a search over all values for k between 1 and 30 was performed. The CGMs have been configured with a range of configurations using bit-depth values of 1, 8, 16 and 32 bits and layers sizes of 2^{16} , 2^{20} , 2^{24} slots with layer counts Λ between 1 and 3, and the datasets were rasterized to a resolution of 8192×8192 pixels. Measuring the representation quality of a Counting GloBiMaps was done by taking the fraction of the erroneous pixels divided by the number of pixels that are greater than zero, this is called the error rate:

$$ERR = \frac{\# \text{ Erroneous Pixels}}{\# \text{ Pixels} > 0}$$

In order to verify the ideal number of hash functions k the error rate was calculated and plotted. As seen in Figures 5, the ideal number of k , that is the k with the lowest error rate achieved, is different for the two datasets.

6.4 Magnitudes of Errors

To investigate the magnitude of errors, the datasets were encoded into the data structure as well as into a hashmap with a key for every point in the grid using a resolution of 8192×192 pixels. The CGMs have been configured with a range of configurations using bit-depth values b_x of 1, 8, 16 and 32 bits and layers sizes m_x of

2^{16} , 2^{20} , 2^{24} with layer counts Λ between 1 and 3 and a number of hash function $k = 8$. After the encoding process, for each point on the grid the difference was taken between the encoded value and the correct value in the hashmap. From this min, max, mean and histogram values were taken, which subsequently have been used to calculate the min, max, mean and std values seen in Figures 6.

Figure 6 a) shows a comparison of error size for different sized datasets, the error sizes here have been normalized to the number of points in the dataset, whereas configurations that can't hold the largest bin sizes or have a compression ratio $cr > 1.0$ where omitted. It is shown that bigger datasets have a smaller relative error which is due to the fact that error size does not scale linearly with the number of points.

Figure 6 b) shows a comparison of error size for different sized CGMs, the error sizes here have been normalized to the number of points in the dataset, whereas configurations that can't hold the largest bin sizes are bigger than 1 gb or have a compression ratio $cr > 1.0$ where omitted. It is clear, that bigger CGMs tend to have smaller error sizes, which is due to compression ratios which are highlighted in Figure 6 c) and d), showing error size on the y-axis and the compression ratio on the x-axis. To calculate the the compression ratios the number of bytes of the filters in the CGMs was divided by the number of points in the dataset times 8, presuming 64 bit or 4 bytes for each element of the coordinate.

Dataset	Resolution	Unique points	Bin Size				
			mean	max	25th %ile	e Median	75th %ile
OSM 200 Mio	4,096	677,700	295	241,790	6	26	103
OSM 200 Mio	8,192	1,738,582	115	96,161	3	12	46
OSM 200 Mio	16,384	4,156,942	48	32,540	2	7	23
OSM 200 Mio	32,768	9,311,414	21	13,072	2	4	13
Twitter 200 Mio	4,096	1,409,058	142	17,020,560	1	1	3
Twitter 200 Mio	8,192	1,983,080	100	16,769,560	1	1	5
Twitter 200 Mio	16,384	2,845,933	70	15,801,333	1	1	5
Twitter 200 Mio	32,768	3,987,981	50	15,769,401	1	1	6

Table 2: Resolution properties

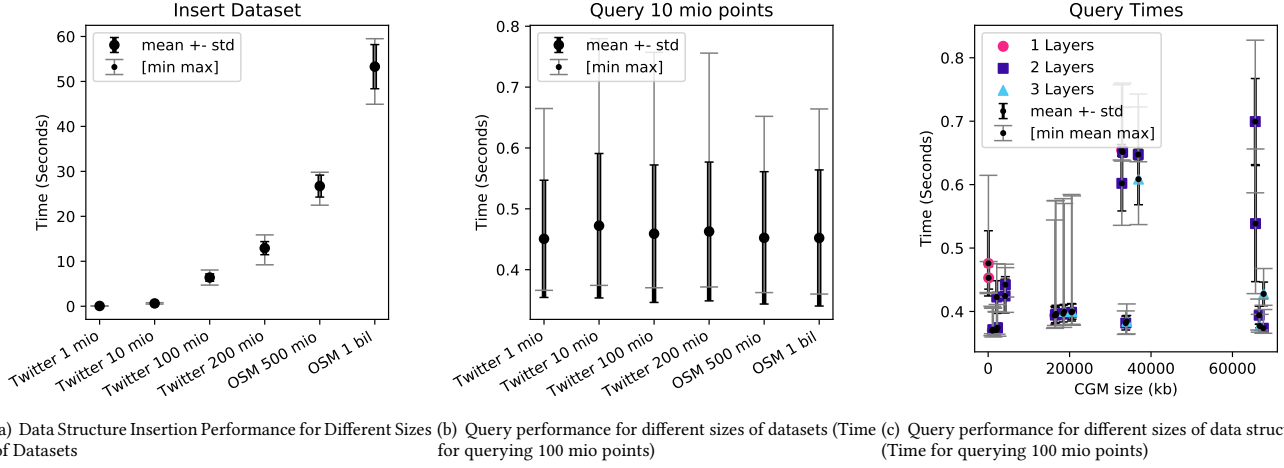


Figure 7: Performance

6.5 Performance Metrics

The evaluation of the performance of the data structure in terms of wall-clock time was performed using a clean and simple C++ implementation running on a common 64 bit system. No micro-optimizations have been done, which leads to some variance in the performance as can be seen in the figures. Overall, we can conclude that the performance of this initial implementation is adequate.

To evaluate the performance of the data structure, insertion of datasets of different sizes was measured for a selection of different configurations. For all tests a grid size of $8096 * 8096$ pixels was used. The CGMs were configured with a range of configurations using bit-depth values b_x of 1, 8, 16 and 32 bits and layers sizes m_x of 2^{16} , 2^{20} and 2^{24} with layer counts Λ between 1 and 3 and a number of hash function $k = 8$.

In a second step, the encoded data structures were queried with a workload of 100 mio uniform randomly distributed points within the grid size. This was done 10 times to generate a minimum, maximum and mean query time.

In figure 7 a) The linear scaling for inserting the data into the data structure is visible. This is not surprising as insertion of a single pixel is in theory dependant size of layers and to so extend the number of layers, this might be apart from noise caused by the operating system and other processes, what is responsible for the scaling of the standard deviation and the min and max values.

In figure 7 b) the time to perform the workload is shown for different sizes of encoded datasets. It is clear that the size of the dataset has no measurable impact, even though some of the queries will stop earlier in less populated structures created by smaller datasets, due to never reaching the upper layers. Other factors are more important here. As only layer numbers from 1 till 3 have been evaluated here, higher layer counts might change this statistic, although higher numbers of layers are not suggested as a strategy.

In figure 7 c) the time to perform the workload is shown for different sizes of CGMs a scaling law is not as clearly identifiable here. Although the slowest times tend to dominate on the right side

with bigger CGMs, neither the size nor the number of layers seem to be a good predictor. Further investigation might be necessary as this could be heavily dependant on the implementation and properties of the hardware. Factors such as page swapping branch prediction and cache misses might play a role here as well as other properties of the filter that have not been investigated at this point.

6.6 Error for Point in Polygon Counts

As mention previously, there are two ways that point in polygon counts can be achieved, the first is using the points of the rasterized polygon and queries the CGM for each point, this can be optimized by precalculating the hash functions for each point. The second way is encoding the rasterized polygon points into a single layer Binary CGM and using the previously described algorithm to calculate the sum. In both cases we can calculate the error for each polygon by taking the difference of the real sum sum_{real} and the sum containing errors sum_{err} and dividing it by the real sum.

$$PERR = \frac{|sum_{real} - sum_{err}|}{sum_{real}}$$

Summarizing the errors over all polygons, an error distribution is constructed and evaluated.

In this experiment, the polygons and pointclouds have been rasterized at a resolution of $16384 * 16384$ pixels. The pointclouds used where the first 1 mio and first 10 mio coordinates of the Twitter dataset. The CGMS where configured with a range of configurations using bit-depth values b_x of 8, 16 and 32 bits and layers sizes m_x of 2^{16} , 2^{20} and 2^{24} with layer counts Λ between 1 and 3 and a number of hash function $k = 8$.

Subsets of the Twitter dataset in the sizes 1 mio, 10 mio and 100 mio, were used to evaluate both polygon datasets.

In Figure 8 a) and b) the performance of both polygon dataset across different point cloud sizes is plotted.

Figure 8 c) the performance of the US Zip Codes dataset is further investigated in relation to the bit-depth of the last showing that higher bit-depths in the last filter lead to smaller error sizes.

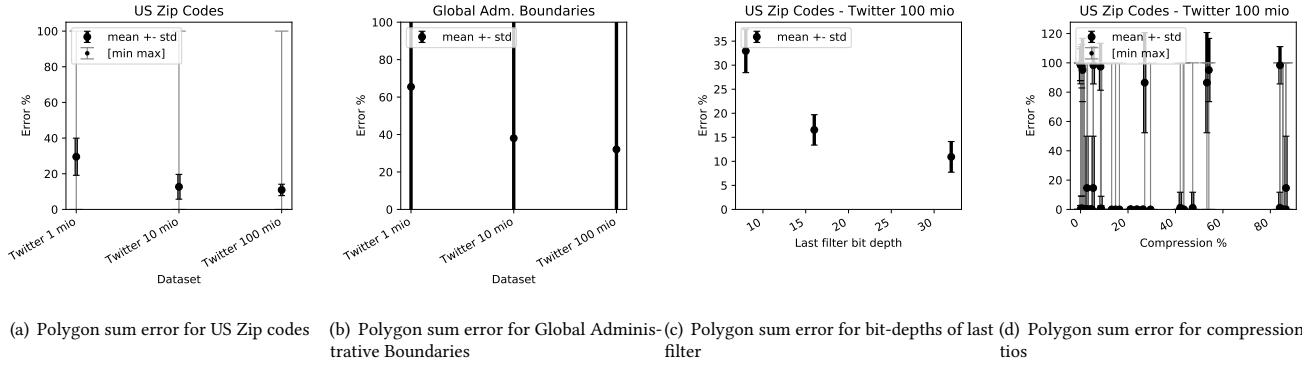


Figure 8: Point in Polygon Error Resistance

Furthermore in Figure 8 c) the impact of compression on error size for the US Zip Codes dataset is displayed. Showing that strong compression does not necessarily mean higher error size, although the likelihood of big errors seems to go down with lower compression.

7 CONCLUSIONS

In this paper, we have introduced Counting GloBiMaps as a novel data structure to deal with large datasets of events or point clouds. It extends on previous work on GloBiMaps in that it first introduces small counters as in the Counting Bloom Filter paradigm and then a layered structure to accommodate counter overflows. To counter the reducing randomness of hash value distributions, we introduce a concept based on increasing the resolution to disambiguate ambiguous cells in later layers. Combined with a small jitter on the input data (unless there is natural noise like in GPS datasets), this provides a versatile and powerful randomized data modeling scheme from its functionality somewhere between frequency count rasters and point clouds. We show that set operations such as Union and Intersection can be lifted from Bloom filters with some work and that even counting queries over complex geometries such as polygons can be efficiently computed. A series of experiments shows that the data structure is practicable and efficient for the tasks it was designed for.

REFERENCES

- [1] [n.d.]. Global LSIB Polygons Detailed - Humanitarian Data Exchange. <https://data.humdata.org/dataset/global-lsib-polygons-detailed>
- [2] J. Aguilar-Saborit, P. Trancoso, V. Muntés-Mulero, and J. L. Larriba-Pey. 2006. Dynamic count filters. *ACM SIGMOD Record* 35, 1 (March 2006), 26–32. <https://doi.org/10.1145/1121995.1122000>
- [3] Tomasz K Baumiller and William I Ausich. 1992. The broken-stick model as a null hypothesis for crinoid stalk taphonomy and as a guide to the distribution of connective tissue in fossils. *Paleobiology* 18, 3 (1992), 288–298.
- [4] Burton H. Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13, 7 (July 1970), 422–426. <https://doi.org/10.1145/362686.362692>
- [5] Andrei Broder and Michael Mitzenmacher. 2004. Network applications of bloom filters: A survey. *Internet mathematics* 1, 4 (2004), 485–509.
- [6] Jehoshua Bruck, Jie Gao, and Anxiao Jiang. 2006. Weighted bloom filter. In *2006 IEEE International Symposium on Information Theory*. IEEE, 2304–2308.
- [7] US Census Bureau. [n.d.]. TIGER/Line Shapefiles. <https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html>
- [8] Jae Min Cho and Kiyoun Choi. 2014. An FPGA implementation of high-throughput key-value store using Bloom filter. In *Technical Papers of 2014 International Symposium on VLSI Design, Automation and Test*. IEEE, 1–4.
- [9] Li Fan, Pei Cao, Jussara Almeida, and Andrei Z. Broder. 2000. Summary cache: a scalable wide-area web cache sharing protocol. *IEEE/ACM transactions on networking* 8, 3 (2000), 281–293.
- [10] D. Ficara, S. Giordano, G. Prociassi, and F. Vitucci. 2008. MultiLayer Compressed Counting Bloom Filters. In *IEEE INFOCOM 2008 - The 27th Conference on Computer Communications*. 311–315. <https://doi.org/10.1109/INFOCOM.2008.71> ISSN: 0743-166X.
- [11] Deke Guo, Jie Wu, Honghui Chen, Ye Yuan, and Xueshan Luo. 2009. The dynamic bloom filters. *IEEE Transactions on Knowledge and Data Engineering* 22, 1 (2009), 120–133.
- [12] Jared Harwayne-Gidansky, Deian Stefan, and Ishaan Dalal. 2009. FPGA-based SoC for real-time network intrusion detection using counting Bloom filters. In *IEEE Southeastcon 2009*. IEEE, 452–458.
- [13] Adam Kirsch and Michael Mitzenmacher. 2006. Distance-sensitive bloom filters. In *2006 Proceedings of the Eighth Workshop on Algorithm Engineering and Experiments (ALENEX)*. SIAM, 41–50.
- [14] Taewhi Lee, Kisung Kim, and Hyoungho Kim. 2012. Join processing using bloom filter in mapreduce. In *Proceedings of the 2012 ACM Research in Applied Computation Symposium*. 100–105.
- [15] Benoit Mandelbrot and Howard M Taylor. 1967. On the distribution of stock price differences. *Operations research* 15, 6 (1967), 1057–1062.
- [16] Loizos Michael, Wolfgang Nejdl, Odysseas Papapetrou, and Wolf Siberski. 2007. Improving distributed join efficiency with extended bloom filter operations. In *21st International Conference on Advanced Information Networking and Applications (AINA'07)*. IEEE, 187–194.
- [17] Michael Mitzenmacher. 2002. Compressed bloom filters. *IEEE/ACM transactions on networking* 10, 5 (2002), 604–612.
- [18] OpenStreetMap contributors. 2017. Planet dump retrieved from <https://planet.osm.org>. <https://www.openstreetmap.org>.
- [19] Odysseas Papapetrou, Wolf Siberski, and Wolfgang Nejdl. 2010. Cardinality estimation and dynamic length adaptation for bloom filters. *Distributed and Parallel Databases* 28, 2 (2010), 119–156.
- [20] Paulo I Prado, Andre Chalom, and Murilo Dantas Miranda. [n.d.]. ML fitting of species rank-abundance distributions. ([n.d.]).
- [21] Anand Ramachandran, Yun Heo, Wen-mei Hwu, Jian Ma, and Deming Chen. 2015. FPGA accelerated DNA error correction. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 1371–1376.
- [22] Peter Ruppel and Axel Küpper. 2014. Geocookie: A space-efficient representation of geographic location sets. *Journal of information processing* 22, 3 (2014), 418–424.
- [23] Martin Werner. 2015. Bacr: Set similarities with lower bounds and application to spatial trajectories. In *Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems*. 1–10.
- [24] Martin Werner. 2019. Globimaps-A probabilistic data structure for in-memory processing of global raster datasets. In *Proceedings of the 27th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*. 3–12.
- [25] Martin Werner. 2021. GloBiMapsAI: An AI-Enhanced Probabilistic Data Structure for Global Raster Datasets. *ACM Transactions on Spatial Algorithms and Systems* 7, 4 (2021), 1–24.
- [26] George Kingsley Zipf. 1945. The meaning-frequency relationship of words. *The Journal of general psychology* 33, 2 (1945), 251–256.